

Binary File Protocol v1

Author: Antara Utane

In this project, bserve is my file server and bcurl is its client. They transfer files over TCP using a compact binary format. The course requires fixed framing, ten numbered headers with a literal fallback, error handling, persistent connections, unknown-frame skipping, and hexdumps. The layouts and limits below are my design choices. **MUST** means a rule both endpoints must follow.

1. Connection and frame header

Both programs use **TCP**. Each message is a frame: an eight-byte header followed by a payload (the message data). Integers are unsigned, meaning zero or positive. Multibyte integers use **big-endian** order: the most significant byte comes first. u8, u16, and u32 mean one, two, and four bytes. Lengths count bytes, not characters. TCP can split or combine frames, so read exactly eight header bytes, then exactly L payload bytes.

Byte offset	Width	Field	Meaning and allowed values
0-3	32 bits	Payload length L	Bytes after the header; 0-16,777,216
4	8 bits	Type	1 = REQUEST; 2 = RESPONSE; others unknown
5	8 bits	Version	MUST be 1
6-7	16 bits	Reserved	MUST be 0

The total frame size is **8 + L**. I chose four bytes for the length because it is simple to encode and leaves room for expansion. The current **16 MiB payload limit** controls memory use. One byte is enough for each of type and version. Two reserved bytes complete the header and leave room for future flags. Unlike HTTP/2's 24/8/8/31 layout, this protocol handles requests in order and needs no stream ID. Only REQUEST and RESPONSE are known types; errors also use RESPONSE.

2. Request and response

Send the fields in the order shown, with no padding or ending markers.

REQUEST (type 1, client -> server):

method:u8 | path_length:u16 | path bytes | header_count:u16 | headers

Method MUST be 1 (GET). The path is UTF-8 text, with a byte length of 1-65,535. Header count is 0-65,535; read exactly that many entries. A request has no body, so any bytes left after its headers make it malformed.

RESPONSE (type 2, server -> client):

status:u16 | header_count:u16 | headers | body bytes

The status is an integer from 100 to 599. My server returns **200** for a file, **400** for a malformed request or unsafe path, **404** for a missing file or a path that is not a regular file, and **500** for an unexpected file error or a response above the payload limit. Each response is final, with no interim responses. Header count is 0-65,535.

After reading the headers, **all remaining payload bytes are the body**. Its length is L minus the bytes used by the status, count, and headers. There is no separate body-length field. Return the file bytes unchanged, including empty and binary files. Error bodies are short UTF-8 messages; their exact wording may vary.

3. How headers are stored

Common header names use one-byte IDs to save space. Other names are sent as text with a length in front.

Numbered entry: name_id:u8 | value_length:u16 | value bytes.

Literal entry: 0:u8 | name_length:u16 | name bytes | value_length:u16 | value bytes.

IDs 1-10 refer to the table below; 11-255 are invalid. ID 0 introduces a literal name: lowercase ASCII letters, digits, or hyphens, with a length of 1-65,535. Values are UTF-8 text, with a byte length of 0-65,535; an empty value is valid. Every field must fit within the payload. A numbered name may also be sent in literal form. Receivers must accept valid literal names they do not recognize. Header order and repeated names have no special meaning.

ID	Name	Normal sender and value
1	host	Client: requested host and port
2	user-agent	Client: bcurl/1
3	accept	Client: */*
4	accept-encoding	Client: identity
5	connection	Client: keep-alive
6	content-type	Server: file type or application/octet-stream; errors text/plain
7	content-length	Server: decimal body byte count
8	server	Server: bserve/1
9	cache-control	Server: no-store
10	x-protocol-version	Server: 1

My client sends IDs 1-5, and my server sends IDs 6-10, including in error responses. Receivers also accept other correctly encoded lists. These headers describe the message; they do not control parsing. The frame length determines the body length, even if content-length says something different. Other header values do not change protocol behavior.

4. File paths, errors, and connections

A path must start with /. The path / means /index.html. For other paths, split on /, ignore empty and . parts, and join the remaining parts under the server root. Reject . . parts, NUL, backslash, colon, or a resolved target outside that root. **Treat percent escapes, query strings, and fragments as literal filename text:** do not decode or interpret them. An invalid or unsafe path returns 400; missing files and directory requests return 404.

A request is malformed if its version or reserved field is wrong, a known type is sent in the wrong direction, the method is not GET, text or header encoding is invalid, a field is missing or extends past the payload, or extra request bytes remain. For a complete frame within the limit, the server reads all L payload bytes, sends one 400, and keeps the connection open. It does not guess where the next frame starts.

If L is above the limit, send 400 if possible, then close without reading the payload. If the peer ends its data partway through a frame (EOF), send 400 if possible and close. A broken connection may prevent a reply. EOF between frames is a normal end. These closing rules are my design choice for oversized or incomplete input. A client receiving malformed input closes and exits with 1; it does not send an error frame.

Send only one request at a time and wait for its response. The server repeats **read frame -> handle/skip -> reply when needed -> read next frame** on the same connection. It keeps the connection open after 200, 404, 500, and recoverable 400. My client makes one connection attempt, sends one request, reads its response, and closes. It must never reconnect or retry.

5. Unknown frames, commands, and hexdumps

A receiver that encounters an unknown frame type MUST skip it cleanly using the frame length. If the header has valid length, version, and reserved fields, discard exactly L bytes in small chunks and read the next frame. Do not reply just because a type is unknown. An unknown frame with L = 0 is valid. Bad headers or incomplete payloads follow the error rules above. A waiting client skips unknown frames until its response arrives.

Run ./bserve ./www 9000 and ./bcurl -v localhost:9000/index.html. On Windows, use python bserve and python bcurl with the same arguments. Client input is host:port/path, with a port from 1-65,535; localhost works for a local demonstration.

The client writes unchanged body bytes to binary stdout. Error messages and -v hexdumps go to stderr. With -v, show every sent and received frame, including skipped frames, with direction, byte offsets, and hex bytes. Label incomplete captures as partial. Exit 0 for 2xx responses and 1 for all failures, including 4xx/5xx, invalid input, malformed responses, and connection errors.

My submission includes both programs, this two-page specification, and an **annotated hexdump of one actual complete request and response**, explaining every field and byte range.